

6. SotilityAdaptiveTokenomics.sol (continued)

Plain Text

```
function _adaptTokenomics() internal {
    // Example adaptation logic:

    // 1. Adjust SUG inflation based on user growth and content creation
    uint256 newInflationRate;
    if (currentState.userGrowthRate > 500) { // >5% growth
        // Increase inflation to reward growth
        newInflationRate = currentParameters.sugInflationRate + 100; //
+1%
    } else {
        // Decrease inflation to reduce token supply growth
        newInflationRate = currentParameters.sugInflationRate > 200 ?
100; // min 1%
        currentParameters.sugInflationRate - 100 :
    }

    // 2. Adjust content rewards based on content creation rate
    uint256 newContentReward;
    if (currentState.contentCreationRate > 1000) { // High content volume
        // Reduce base rewards to control token issuance
        newContentReward = (currentParameters.sugContentReward * 90) /
100; // 90% of current
    } else {
        // Increase rewards to incentivize more content
        newContentReward = (currentParameters.sugContentReward * 110) /
100; // 110% of current
    }

    // 3. Adjust SST minting threshold based on TVL
    uint256 newMintingThreshold;
    if (currentState.totalValueLocked > 100000000 * 1e6) { // >$10M TVL
        // Lower threshold for established ecosystem
        newMintingThreshold = (currentParameters.sstMintingThreshold *
95) / 100; // 95% of current
    } else {
        // Higher threshold for smaller ecosystem
        newMintingThreshold = (currentParameters.sstMintingThreshold *
105) / 100; // 105% of current
    }

    // Update parameters
    currentParameters = TokenomicParameters({
        sugInflationRate: newInflationRate,
        sugContentReward: newContentReward,
```

```

        sotDividendRatio: currentParameters.sotDividendRatio, // keep
unchanged for now
        sstMintingThreshold: newMintingThreshold,
        veTokenBoostMax: currentParameters.veTokenBoostMax, // keep
unchanged for now
        timestamp: block.timestamp
    });

    parameterHistory.push(currentParameters);

    emit TokenomicParametersUpdated(block.timestamp,
parameterHistory.length - 1);
}

/**
 * @dev Get the current SUG inflation rate
 */
function getCurrentInflationRate() external view returns (uint256) {
    return currentParameters.sugInflationRate;
}

/**
 * @dev Get the current content reward base amount
 */
function getCurrentContentReward() external view returns (uint256) {
    return currentParameters.sugContentReward;
}

/**
 * @dev Get the current SOT dividend ratio
 */
function getCurrentDividendRatio() external view returns (uint256) {
    return currentParameters.sotDividendRatio;
}

/**
 * @dev Get the current SST minting threshold
 */
function getCurrentMintingThreshold() external view returns (uint256) {
    return currentParameters.sstMintingThreshold;
}

/**
 * @dev Get historical ecosystem states
 * @param index Index in the history array
 */
function getHistoricalState(uint256 index) external view returns
(EcosystemState memory) {

```

```

        require(index < stateHistory.length, "Index out of bounds");
        return stateHistory[index];
    }

    /**
     * @dev Get historical tokenomic parameters
     * @param index Index in the history array
     */
    function getHistoricalParameters(uint256 index) external view returns
(TokenomicParameters memory) {
        require(index < parameterHistory.length, "Index out of bounds");
        return parameterHistory[index];
    }
}

```

AI and Oracle Contracts

1. AIOracleManager.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title AIOracleManager
 * @dev Register, activate, and manage trusted AI verifier agents for
Sotility modules.
 */
contract AIOracleManager is AccessControl {
    bytes32 public constant ORACLE_ADMIN_ROLE =
keccak256("ORACLE_ADMIN_ROLE");

    struct Oracle {
        string name;
        string version;
        string description;
        bool active;
    }

    mapping(address => Oracle) public oracles;
    address[] public oracleList;

    event OracleRegistered(address indexed oracle, string name, string
version);

```

```

event OracleStatusChanged(address indexed oracle, bool active);

constructor() {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(ORACLE_ADMIN_ROLE, msg.sender);
}

function registerOracle(
    address oracle,
    string calldata name,
    string calldata version,
    string calldata description
) external onlyRole(ORACLE_ADMIN_ROLE) {
    require(oracles[oracle].active == false, "Already registered");

    oracles[oracle] = Oracle({
        name: name,
        version: version,
        description: description,
        active: true
    });

    oracleList.push(oracle);

    emit OracleRegistered(oracle, name, version);
}

function setOracleStatus(address oracle, bool active) external
onlyRole(ORACLE_ADMIN_ROLE) {
    require(bytes(oracles[oracle].name).length > 0, "Not found");

    oracles[oracle].active = active;

    emit OracleStatusChanged(oracle, active);
}

function isActiveOracle(address oracle) public view returns (bool) {
    return oracles[oracle].active;
}

function getOracle(address oracle) public view returns (Oracle memory) {
    return oracles[oracle];
}

function getAllOracles() external view returns (address[] memory) {
    return oracleList;
}

```

```
}  
}
```

2. MultiOracleAggregator.sol

Plain Text

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.20;  
import "@openzeppelin/contracts/access/AccessControl.sol";  
  
/**  
 * @title MultiOracleAggregator  
 * @dev Aggregates data from multiple AI oracles to prevent oracle  
manipulation  
 */  
contract MultiOracleAggregator is AccessControl {  
    bytes32 public constant ORACLE_ADMIN_ROLE =  
keccak256("ORACLE_ADMIN_ROLE");  
  
    struct OracleData {  
        uint256 score;  
        uint256 timestamp;  
        bool submitted;  
    }  
  
    struct ContentScore {  
        mapping(address => OracleData) oracleScores;  
        address[] oracles;  
        uint256 finalScore;  
        bool finalized;  
    }  
  
    mapping(uint256 => ContentScore) public contentScores;  
    mapping(address => bool) public authorizedOracles;  
    uint256 public minimumOracleResponses;  
    uint256 public scoreDeviationThreshold; // in basis points (e.g., 1000 =  
10%)  
  
    event OracleAuthorized(address indexed oracle);  
    event OracleDeauthorized(address indexed oracle);  
    event ScoreSubmitted(uint256 indexed contentId, address indexed oracle,  
uint256 score);  
    event ScoreFinalized(uint256 indexed contentId, uint256 finalScore);  
  
    constructor(uint256 _minimumOracleResponses, uint256  
_scoreDeviationThreshold) {
```

```

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(ORACLE_ADMIN_ROLE, msg.sender);
        minimumOracleResponses = _minimumOracleResponses;
        scoreDeviationThreshold = _scoreDeviationThreshold;
    }

    function authorizeOracle(address oracle) external
    onlyRole(ORACLE_ADMIN_ROLE) {
        authorizedOracles[oracle] = true;
        emit OracleAuthorized(oracle);
    }

    function deauthorizeOracle(address oracle) external
    onlyRole(ORACLE_ADMIN_ROLE) {
        authorizedOracles[oracle] = false;
        emit OracleDeauthorized(oracle);
    }

    function submitScore(uint256 contentId, uint256 score) external {
        require(authorizedOracles[msg.sender], "Not an authorized oracle");
        require(score <= 100, "Score must be between 0-100");
        require(!contentScores[contentId].finalized, "Score already
finalized");

        require(!contentScores[contentId].oracleScores[msg.sender].submitted,
"Oracle already submitted");

        ContentScore storage content = contentScores[contentId];
        content.oracleScores[msg.sender] = OracleData({
            score: score,
            timestamp: block.timestamp,
            submitted: true
        });
        content.oracles.push(msg.sender);

        emit ScoreSubmitted(contentId, msg.sender, score);

        if (content.oracles.length >= minimumOracleResponses) {
            _attemptFinalization(contentId);
        }
    }

    function _attemptFinalization(uint256 contentId) internal {
        ContentScore storage content = contentScores[contentId];

        // Calculate average score
        uint256 totalScore;
        for (uint i = 0; i < content.oracles.length; i++) {

```

```

        totalScore += content.oracleScores[content.oracles[i]].score;
    }
    uint256 averageScore = totalScore / content.oracles.length;

    // Check for large deviations
    bool hasLargeDeviation = false;
    for (uint i = 0; i < content.oracles.length; i++) {
        uint256 oracleScore =
content.oracleScores[content.oracles[i]].score;
        uint256 deviation;

        if (oracleScore > averageScore) {
            deviation = ((oracleScore - averageScore) * 10000) /
averageScore;
        } else {
            deviation = ((averageScore - oracleScore) * 10000) /
averageScore;
        }

        if (deviation > scoreDeviationThreshold) {
            hasLargeDeviation = true;
            break;
        }
    }

    if (!hasLargeDeviation) {
        content.finalScore = averageScore;
        content.finalized = true;
        emit ScoreFinalized(contentId, averageScore);
    }
}

function getScoreStatus(uint256 contentId) external view returns (
    uint256 oracleCount,
    bool isFinalized,
    uint256 finalScore
) {
    ContentScore storage content = contentScores[contentId];
    return (
        content.oracles.length,
        content.finalized,
        content.finalScore
    );
}

function forceFinalize(uint256 contentId) external
onlyRole(ORACLE_ADMIN_ROLE) {
    ContentScore storage content = contentScores[contentId];

```

```

        require(!content.finalized, "Already finalized");
        require(content.oracles.length >= minimumOracleResponses, "Not
enough oracles");

        uint256 totalScore;
        for (uint i = 0; i < content.oracles.length; i++) {
            totalScore += content.oracleScores[content.oracles[i]].score;
        }
        uint256 averageScore = totalScore / content.oracles.length;

        content.finalScore = averageScore;
        content.finalized = true;

        emit ScoreFinalized(contentId, averageScore);
    }
}

```

Identity and Reputation Contracts

1. SotilityZKIdentity.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityZKIdentity
 * @dev Enables zero-knowledge proofs of reputation without revealing
specific data
 */
contract SotilityZKIdentity is AccessControl {
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");
    bytes32 public constant ISSUER_ROLE = keccak256("ISSUER_ROLE");

    struct ZKProof {
        bytes32 proofHash;
        uint256 timestamp;
        bool verified;
    }

    // Mapping of unique identifiers to their ZK proofs
    mapping(bytes32 => ZKProof) public proofs;

    // Mapping of user addresses to their proof identifiers

```

```

mapping(address => bytes32[]) public userProofs;

event ProofIssued(address indexed user, bytes32 indexed proofId);
event ProofVerified(bytes32 indexed proofId, bool success);

constructor() {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}

/**
 * @dev Issue a new ZK proof for a user
 * @param user Address of the user
 * @param proofId Unique identifier for the proof
 * @param proofHash Hash of the ZK proof data
 */
function issueProof(address user, bytes32 proofId, bytes32 proofHash)
external onlyRole(ISSUER_ROLE) {
    require(proofs[proofId].timestamp == 0, "Proof ID already exists");

    proofs[proofId] = ZKProof({
        proofHash: proofHash,
        timestamp: block.timestamp,
        verified: false
    });

    userProofs[user].push(proofId);

    emit ProofIssued(user, proofId);
}

/**
 * @dev Verify a ZK proof
 * @param proofId Identifier of the proof to verify
 * @param proof ZK proof data to validate
 */
function verifyProof(bytes32 proofId, bytes calldata proof) external
onlyRole(VERIFIER_ROLE) returns (bool) {
    require(proofs[proofId].timestamp > 0, "Proof does not exist");

    // In a real implementation, this would involve complex ZK proof
    verification
    // For this example, we're using a simplified placeholder
    bool success = keccak256(proof) == proofs[proofId].proofHash;

    if (success) {
        proofs[proofId].verified = true;
    }
}

```

```

        emit ProofVerified(proofId, success);
        return success;
    }

    /**
     * @dev Check if a user has a verified proof without revealing the proof
data
     * @param user Address of the user
     * @param proofCategory Category of proof to check (e.g., "AGE_OVER_18",
"CREDIT_SCORE_ABOVE_700")
     */
    function hasVerifiedProof(address user, string calldata proofCategory)
external view returns (bool) {
        bytes32 categoryHash = keccak256(abi.encodePacked(proofCategory));

        bytes32[] memory userProofIds = userProofs[user];
        for (uint i = 0; i < userProofIds.length; i++) {
            bytes32 proofId = userProofIds[i];

            // Check if this proof is of the requested category and is
verified
            if (proofId & categoryHash == categoryHash &&
proofs[proofId].verified) {
                return true;
            }
        }

        return false;
    }

    /**
     * @dev Get all proof IDs for a user
     * @param user Address of the user
     */
    function getUserProofIds(address user) external view returns (bytes32[]
memory) {
        return userProofs[user];
    }
}

```

2. SotilityCrossChainIdentity.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

```

```

/**
 * @title SotilityCrossChainIdentity
 * @dev Aggregates reputation and identity information across multiple
blockchains
 */
contract SotilityCrossChainIdentity is AccessControl {
    bytes32 public constant BRIDGE_ROLE = keccak256("BRIDGE_ROLE");
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");

    struct ChainIdentity {
        uint256 chainId;
        address identityContract;
        bytes32 identityId;
        uint256 reputationScore;
        uint256 lastUpdated;
        bool verified;
    }

    // User's identity across multiple chains
    mapping(address => mapping(uint256 => ChainIdentity)) public
chainIdentities;
    mapping(address => uint256[]) public userChains;

    // Aggregated reputation score across all chains
    mapping(address => uint256) public aggregatedReputation;

    event IdentityLinked(address indexed user, uint256 chainId, bytes32
identityId);
    event ReputationUpdated(address indexed user, uint256 chainId, uint256
newScore);
    event AggregatedScoreUpdated(address indexed user, uint256 newScore);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    /**
     * @dev Link an identity from another chain to this user
     * @param user Address of the user
     * @param chainId ID of the blockchain (e.g., Ethereum = 1, Polygon =
137)
     * @param identityContract Address of the identity contract on that chain
     * @param identityId Unique identifier of the user on that chain
     * @param initialScore Initial reputation score from that chain
     */
    function linkChainIdentity(
        address user,

```

```

        uint256 chainId,
        address identityContract,
        bytes32 identityId,
        uint256 initialScore
    ) external onlyRole(BRIDGE_ROLE) {
        require(chainIdentities[user][chainId].lastUpdated == 0, "Chain
identity already linked");

        chainIdentities[user][chainId] = ChainIdentity({
            chainId: chainId,
            identityContract: identityContract,
            identityId: identityId,
            reputationScore: initialScore,
            lastUpdated: block.timestamp,
            verified: false
        });

        userChains[user].push(chainId);

        emit IdentityLinked(user, chainId, identityId);

        _updateAggregatedScore(user);
    }

    /**
     * @dev Update reputation score from another chain
     * @param user Address of the user
     * @param chainId ID of the blockchain
     * @param newScore Updated reputation score
     */
    function updateReputationScore(
        address user,
        uint256 chainId,
        uint256 newScore
    ) external onlyRole(BRIDGE_ROLE) {
        require(chainIdentities[user][chainId].lastUpdated > 0, "Chain
identity not linked");

        chainIdentities[user][chainId].reputationScore = newScore;
        chainIdentities[user][chainId].lastUpdated = block.timestamp;

        emit ReputationUpdated(user, chainId, newScore);

        _updateAggregatedScore(user);
    }

    /**
     * @dev Verify an identity on another chain

```

```

    * @param user Address of the user
    * @param chainId ID of the blockchain
    */
    function verifyChainIdentity(address user, uint256 chainId) external
onlyRole(VERIFIER_ROLE) {
        require(chainIdentities[user][chainId].lastUpdated > 0, "Chain
identity not linked");

        chainIdentities[user][chainId].verified = true;

        _updateAggregatedScore(user);
    }

    /**
    * @dev Calculate and update the aggregated reputation score
    * @param user Address of the user
    */
    function _updateAggregatedScore(address user) internal {
        uint256[] memory chains = userChains[user];
        uint256 totalScore;
        uint256 verifiedChains;

        for (uint i = 0; i < chains.length; i++) {
            ChainIdentity storage identity = chainIdentities[user]
[chains[i]];

            if (identity.verified) {
                totalScore += identity.reputationScore;
                verifiedChains++;
            }
        }

        // Calculate weighted average of verified chain scores
        if (verifiedChains > 0) {
            aggregatedReputation[user] = totalScore / verifiedChains;
        } else {
            aggregatedReputation[user] = 0;
        }

        emit AggregatedScoreUpdated(user, aggregatedReputation[user]);
    }

    /**
    * @dev Get all chains a user has linked identities on
    * @param user Address of the user
    */
    function getUserChains(address user) external view returns (uint256[]
memory) {

```

```

        return userChains[user];
    }

    /**
     * @dev Get a user's aggregated reputation score
     * @param user Address of the user
     */
    function getAggregatedReputation(address user) external view returns
    (uint256) {
        return aggregatedReputation[user];
    }
}

```

3. SotilityProofOfPersonhood.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityProofOfPersonhood
 * @dev Sybil-resistant identity verification without revealing personal data
 */
contract SotilityProofOfPersonhood is AccessControl {
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");
    bytes32 public constant REVOKER_ROLE = keccak256("REVOKER_ROLE");

    enum VerificationStatus { None, Pending, Verified, Revoked }

    struct PersonhoodCredential {
        VerificationStatus status;
        uint256 verifiedAt;
        uint256 expiresAt;
        bytes32 credentialHash;
    }

    // Mapping of addresses to their personhood status
    mapping(address => PersonhoodCredential) public credentials;

    // Total verified accounts
    uint256 public totalVerified;

    // Events
    event VerificationRequested(address indexed user, bytes32 requestId);
    event VerificationCompleted(address indexed user);
}

```

```

event VerificationRevoked(address indexed user, string reason);
event CredentialRenewed(address indexed user, uint256 newExpiry);

constructor() {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(VERIFIER_ROLE, msg.sender);
    _grantRole(REVOKER_ROLE, msg.sender);
}

/**
 * @dev Request verification of personhood
 * @param credentialHash Hash of the verification credential
(implementation-specific)
 */
function requestVerification(bytes32 credentialHash) external {
    require(credentials[msg.sender].status == VerificationStatus.None,
"Verification already exists");

    credentials[msg.sender] = PersonhoodCredential({
        status: VerificationStatus.Pending,
        verifiedAt: 0,
        expiresAt: 0,
        credentialHash: credentialHash
    });

    emit VerificationRequested(msg.sender, credentialHash);
}

/**
 * @dev Complete verification process (by authorized verifier)
 * @param user Address of the user to verify
 * @param expiryDuration How long the verification remains valid (in
seconds)
 */
function completeVerification(address user, uint256 expiryDuration)
external onlyRole(VERIFIER_ROLE) {
    require(credentials[user].status == VerificationStatus.Pending,
"Invalid verification status");

    uint256 expiryTime = block.timestamp + expiryDuration;

    credentials[user].status = VerificationStatus.Verified;
    credentials[user].verifiedAt = block.timestamp;
    credentials[user].expiresAt = expiryTime;

    totalVerified++;

    emit VerificationCompleted(user);
}

```

```

}

/**
 * @dev Revoke verification (by authorized revoker)
 * @param user Address of the user to revoke
 * @param reason Reason for revocation
 */
function revokeVerification(address user, string calldata reason)
external onlyRole(REVOKER_ROLE) {
    require(credentials[user].status == VerificationStatus.Verified,
"Not verified");

    credentials[user].status = VerificationStatus.Revoked;

    if (totalVerified > 0) {
        totalVerified--;
    }

    emit VerificationRevoked(user, reason);
}

/**
 * @dev Renew verification before expiry
 * @param expiryDuration New duration to add to current timestamp
 */
function renewVerification(uint256 expiryDuration) external {
    require(credentials[msg.sender].status ==
VerificationStatus.Verified, "Not verified");
    require(block.timestamp <= credentials[msg.sender].expiresAt,
"Verification expired");

    uint256 newExpiry = block.timestamp + expiryDuration;
    credentials[msg.sender].expiresAt = newExpiry;

    emit CredentialRenewed(msg.sender, newExpiry);
}

/**
 * @dev Check if an address is a verified person
 * @param user Address to check
 */
function isVerifiedPerson(address user) external view returns (bool) {
    return (
        credentials[user].status == VerificationStatus.Verified &&
        block.timestamp <= credentials[user].expiresAt
    );
}

```

```

/**
 * @dev Get verification details
 * @param user Address to check
 */
function getVerificationDetails(address user) external view returns (
    VerificationStatus status,
    uint256 verifiedAt,
    uint256 expiresAt,
    bool isValid
) {
    PersonhoodCredential memory cred = credentials[user];

    bool valid = (
        cred.status == VerificationStatus.Verified &&
        block.timestamp <= cred.expiresAt
    );

    return (
        cred.status,
        cred.verifiedAt,
        cred.expiresAt,
        valid
    );
}
}

```

Security and Risk Management Contracts

1. SotilityEmergencyShutdown.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityEmergencyShutdown
 * @dev Circuit breaker pattern for Sotility ecosystem in case of attacks
 */
contract SotilityEmergencyShutdown is AccessControl {
    bytes32 public constant GUARDIAN_ROLE = keccak256("GUARDIAN_ROLE");
    bytes32 public constant RECOVERY_ROLE = keccak256("RECOVERY_ROLE");

    mapping(address => bool) public pausedContracts;
    bool public systemWideEmergency;
}

```

```

uint256 public emergencyTimelock;
uint256 public constant EMERGENCY_DELAY = 24 hours;

event ContractPaused(address indexed contractAddress);
event ContractUnpaused(address indexed contractAddress);
event SystemWideEmergencyActivated();
event SystemWideEmergencyDeactivated();
event EmergencyTimelockInitiated(uint256 activationTime);

constructor(address[] memory guardians, address[] memory recoveryTeam) {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);

    for (uint i = 0; i < guardians.length; i++) {
        _grantRole(GUARDIAN_ROLE, guardians[i]);
    }

    for (uint i = 0; i < recoveryTeam.length; i++) {
        _grantRole(RECOVERY_ROLE, recoveryTeam[i]);
    }
}

function initiateEmergencyTimelock() external onlyRole(GUARDIAN_ROLE) {
    require(emergencyTimelock == 0, "Timelock already initiated");
    emergencyTimelock = block.timestamp + EMERGENCY_DELAY;
    emit EmergencyTimelockInitiated(emergencyTimelock);
}

function activateSystemWideEmergency() external onlyRole(GUARDIAN_ROLE) {
    require(block.timestamp >= emergencyTimelock, "Emergency timelock
not expired");
    systemWideEmergency = true;
    emit SystemWideEmergencyActivated();
}

function deactivateSystemWideEmergency() external
onlyRole(RECOVERY_ROLE) {
    systemWideEmergency = false;
    emergencyTimelock = 0;
    emit SystemWideEmergencyDeactivated();
}

function pauseContract(address contractAddress) external
onlyRole(GUARDIAN_ROLE) {
    pausedContracts[contractAddress] = true;
    emit ContractPaused(contractAddress);
}

function unpauseContract(address contractAddress) external

```

```

onlyRole(RECOVERY_ROLE) {
    pausedContracts[contractAddress] = false;
    emit ContractUnpaused(contractAddress);
}

function isContractPaused(address contractAddress) external view returns
(bool) {
    return pausedContracts[contractAddress] || systemWideEmergency;
}
}

```

2. SotilityInsurance.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

/**
 * @title SotilityInsurance
 * @dev Protocol insurance fund for protecting against exploits and failures
 */
contract SotilityInsurance is AccessControl, ReentrancyGuard {
    bytes32 public constant CLAIM_APPROVER_ROLE =
keccak256("CLAIM_APPROVER_ROLE");
    bytes32 public constant PREMIUM_ADJUSTER_ROLE =
keccak256("PREMIUM_ADJUSTER_ROLE");

    enum PolicyStatus { Active, Expired, Claimed }
    enum ClaimStatus { None, Pending, Approved, Rejected }

    struct Policy {
        address holder;
        uint256 coverageAmount;
    }
}

```